

Università degli Studi di Torino
Corso di Laurea Magistrale in Informatica

Tecnologie del Linguaggio Naturale

Parte 2 – Prof. Daniele Radicioni

Lab 4

Modellazione del Linguaggio con N-grammi
Confronto tra linguaggio dei social media e linguaggio letterario

Gruppo di lavoro:

Studente	Matricola
Alessandro Olivero	915069
Samuele Iudice	1235245
Andrea Franco	1226739

Docente: Prof. Daniele Radicioni

Anno Accademico: 2025/2026

Indice

1	Introduzione	2
2	Background Teorico	2
2.1	N-grammi	2
2.2	Modello MLE (Maximum Likelihood Estimation)	2
2.3	Smoothing di Laplace (Add-1)	3
2.4	Perplexity	3
2.5	Log-likelihood	3
2.6	Type-Token Ratio (TTR)	3
3	Dataset e Pre-elaborazione	4
3.1	Dataset	4
3.2	Pipeline di pulizia	4
4	Analisi Statistica dei Corpus	4
4.1	Tokenizzazione e vocabolario	4
4.2	Rimozione delle stopwords e analisi dei bigrammi	5
5	Addestramento dei Modelli Linguistici	5
5.1	Preparazione e fitting	5
5.2	Probabilità condizionali	6
6	Generazione di Testo	6
7	Perplexity Cross-Domain	7
7.1	Configurazione dell'esperimento	7
7.2	Limiti della perplexity come classificatore	7
8	Classificatore con Log-Likelihood	7
9	MLE vs. Laplace: confronto diretto	8
10	Confronto $n = 1, 2, 3$	9
11	Analisi Linguistica Comparata	9
11.1	Type-Token Ratio	9
11.2	Lunghezza media delle frasi	9
12	Risultati e Discussione	10
12.1	Osservazioni principali	10
13	Conclusioni	10

1 Introduzione

Questo laboratorio ha l'obiettivo di esplorare la **modellazione del linguaggio** tramite N-grammi, confrontando due domini linguistici molto diversi tra loro:

- **Linguaggio Twitter**: messaggi brevi, informali, ricchi di slang, abbreviazioni ed emoji.
- **Linguaggio letterario**: testo formale tratto dal romanzo *Emma* di Jane Austen (1816), caratterizzato da frasi lunghe e sintassi elaborata.

L'analisi copre i seguenti aspetti:

1. Statistiche descrittive sui corpus (vocabolario, TTR, lunghezza frasi).
2. Estrazione e analisi di bigrammi e trigrammi.
3. Addestramento di modelli linguistici MLE e Laplace.
4. Generazione automatica di testo.
5. Classificazione di frasi tramite perplexity e log-likelihood.

2 Background Teorico

2.1 N-grammi

Un **N-gramma** è una sequenza contigua di N token (parole) estratta da un testo. I casi più comuni sono:

- **Unigramma** ($N = 1$): singola parola.
- **Bigramma** ($N = 2$): coppia di parole consecutive.
- **Trigramma** ($N = 3$): tripletta di parole consecutive.

Gli N-grammi sono la base dei **modelli linguistici statistici**: stimano la probabilità di una parola dato il contesto delle $N-1$ parole precedenti.

2.2 Modello MLE (Maximum Likelihood Estimation)

Il modello MLE stima la probabilità condizionale con la frequenza relativa osservata nel corpus:

$$P_{\text{MLE}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

dove $C(\cdot)$ indica il conteggio nel corpus. Il problema principale del MLE è che assegna probabilità **zero** a qualsiasi sequenza non vista durante il training: basta una parola fuori dal vocabolario per rendere l'intera frase impossibile secondo il modello.

2.3 Smoothing di Laplace (Add-1)

Per ovviare al problema dello zero, lo **smoothing di Laplace** aggiunge 1 a ogni conteggio:

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|}$$

dove $|V|$ è la dimensione del vocabolario. In questo modo nessuna sequenza ha probabilità nulla, al costo di ridistribuire parte della probabilità verso eventi non osservati.

2.4 Perplexity

La **perplexity** misura quanto un modello è “sorpreso” da un testo di test. Un modello buono assegna alta probabilità al testo, producendo bassa perplexity.

$$\text{PP}(W) = P(w_1 w_2 \cdots w_N)^{-\frac{1}{N}} = \left(\prod_{i=1}^N \frac{1}{P(w_i \mid w_{i-1})} \right)^{\frac{1}{N}}$$

Più bassa è la perplexity, migliore è il modello per quel testo.

2.5 Log-likelihood

La **log-likelihood** di una sequenza misura quanto il modello “conosce” quel testo. Si calcola come somma dei logaritmi delle probabilità dei singoli token:

$$\ell(W) = \sum_{i=1}^N \log P(w_i \mid w_{i-1})$$

Il valore è sempre ≤ 0 : più è vicino a zero (meno negativo), meglio il modello descrive il testo. Rispetto alla perplexity è numericamente più stabile ed è diretta da usare come classificatore: si assegna un testo al dominio che gli assegna log-likelihood più alta.

2.6 Type-Token Ratio (TTR)

Il **TTR** misura la ricchezza lessicale di un testo:

$$\text{TTR} = \frac{|\text{Types}|}{|\text{Tokens}|}$$

dove *Types* sono le parole uniche e *Tokens* sono tutte le occorrenze. Valori alti indicano vocabolario vario; valori bassi indicano ripetitività.

3 Dataset e Pre-elaborazione

3.1 Dataset

- **Twitter:** corpus `twitter_samples` di NLTK, composto da 5 000 tweet positivi e 5 000 tweet negativi (10 000 tweet in totale).
- **Letterario:** testo integrale di *Emma* di Jane Austen (1816), disponibile nel corpus `gutenberg` di NLTK.

3.2 Pipeline di pulizia

I due corpus richiedono trattamenti diversi a causa della natura del linguaggio.

Pulizia dei tweet e del testo letterario

```
1 def pulisci_tweet(testo):
2     testo = re.sub(r'http\S+', '', testo)           # rimuove URL
3     testo = re.sub(r'@\w+', '', testo)             # rimuove menzioni
4     testo = re.sub(r'#', '', testo)                # rimuove simbolo hashtag
5     testo = re.sub(r'[^a-zA-Z\s]', '', testo)      # solo lettere
6     testo = testo.lower().strip()
7     return testo
8
9 def pulisci_letterario(testo):
10    testo = re.sub(r'[^a-zA-Z\s]', '', testo)
11    testo = testo.lower().strip()
12    return testo
```

Per i tweet si rimuovono URL, menzioni e hashtag prima di normalizzare il testo. Per il testo letterario si rimuovono solo i caratteri non alfabetici, mantenendo la struttura originale.

4 Analisi Statistica dei Corpus

4.1 Tokenizzazione e vocabolario

Tokenizzazione e calcolo del vocabolario

```
1 tokens_tweet = word_tokenize(testo_tweet_unito)
2 tokens_lett  = word_tokenize(testo_lett_pulito)
3
4 vocab_tweet_set = set(tokens_tweet)
5 vocab_lett_set  = set(tokens_lett)
```

Tabella 1: Statistiche descrittive dei corpus

Statistica	Twitter	Letterario
Token totali	~120 000	~160 000
Parole uniche (types)	~20 000	~8 000
TTR	~0.17	~0.05
Lunghezza media frase	~10 token	~22 token

Il TTR più alto di Twitter riflette l'enorme varietà lessicale (slang, hashtag trasformati in parole, emoticon testuali), mentre il testo letterario ha un vocabolario più ristretto ma frasi molto più lunghe.

4.2 Rimozione delle stopwords e analisi dei bigrammi

Rimozione stopwords e calcolo bigrammi/trigrammi

```
1 stop = set(stopwords.words('english'))
2
3 tokens_tweet_filtrati = [t for t in tokens_tweet if t not in stop]
4 tokens_lett_filtrati = [t for t in tokens_lett if t not in stop]
5
6 bi_tweet_filtrati = Counter(ngrams(tokens_tweet_filtrati, 2))
7 bi_lett_filtrati = Counter(ngrams(tokens_lett_filtrati, 2))
```

Rimuovendo le stopwords emergono i bigrammi semanticamente significativi di ciascun dominio:

- **Twitter:** bigrammi legati a emozioni (*feel good, love much*), auguri (*happy birthday*) e relazioni (*miss much*).
- **Letterario:** bigrammi tipici del romanzo vittoriano (*mr knightley, miss woodhouse, emma thought*).

5 Addestramento dei Modelli Linguistici

5.1 Preparazione e fitting

NLTK fornisce `padded_everygram_pipeline`, che aggiunge automaticamente i token di inizio (`<s>`) e fine (`</s>`) frase e genera tutti gli N-grammi fino a n :

Addestramento modelli MLE e Laplace per bigrammi ($n = 2$)

```
1 n = 2
2
3 # Modelli MLE: per generazione e probabilit  condizionali
4 train_tw_mle, vocab_tw_mle = padded_everygram_pipeline(n, [tokens_tweet])
5 modello_tweet = MLE(n)
6 modello_tweet.fit(train_tw_mle, vocab_tw_mle)
7
8 train_lt_mle, vocab_lt_mle = padded_everygram_pipeline(n, [tokens_lett])
9 modello_lett = MLE(n)
10 modello_lett.fit(train_lt_mle, vocab_lt_mle)
11
12 # Modelli Laplace: per perplexity (gestisce parole mai viste)
13 train_tw_lp, vocab_tw_lp = padded_everygram_pipeline(n, [tokens_tweet])
14 modello_tweet_lp = Laplace(n)
15 modello_tweet_lp.fit(train_tw_lp, vocab_tw_lp)
16
17 train_lt_lp, vocab_lt_lp = padded_everygram_pipeline(n, [tokens_lett])
18 modello_lett_lp = Laplace(n)
19 modello_lett_lp.fit(train_lt_lp, vocab_lt_lp)
```

Si addestrano due varianti per ciascun corpus:

- **MLE**: usato per la generazione di testo e il calcolo delle probabilità condizionali.
- **Laplace**: usato per la perplexity, poiché MLE assegna probabilità 0 a parole mai viste, rendendo la perplexity infinita.

5.2 Probabilità condizionali

Calcolo probabilità condizionali con MLE

```
1 # Modello Twitter
2 print(modello_tweet.score('love', ['i']))      # P('love' | 'i')
3 print(modello_tweet.score('happy', ['so']))    # P('happy' | 'so')
4
5 # Modello Letterario
6 print(modello_lett.score('mr', ['said']))      # P('mr' | 'said')
7 print(modello_lett.score('been', ['had']))     # P('been' | 'had')
```

Le probabilità condizionali riflettono il registro linguistico: il modello Twitter assegna alta probabilità a *love* dopo *i*, mentre il letterario assegna alta probabilità a *mr* dopo *said* e a *been* dopo *had*, strutture tipiche della prosa ottocentesca.

6 Generazione di Testo

Generazione di testo con filtraggio degli artefatti

```
1 def genera_testo(modello, n_parole=30):
2     testo_raw = modello.generate(n_parole, random_seed=42)
3
4     testo_filtrato = []
5     for token in testo_raw:
6         # scarta token speciali del padding
7         if token in ['<s>', '</s>', '<UNK>']:
8             continue
9         # scarta lettere singole (artefatti), tranne 'i' e 'a'
10        if len(token) == 1 and token not in ['i', 'a']:
11            continue
12        testo_filtrato.append(token)
13
14    return ' '.join(testo_filtrato)
15
16 # Esempi di output
17 print(genera_testo(modello_tweet, 40))      # stile Twitter
18 print(genera_testo(modello_lett, 40))      # stile letterario
```

Il metodo `generate()` campiona parola per parola usando le probabilità del bigramma. I token `<s>`, `</s>` e `<UNK>` sono artefatti del padding e vengono rimossi. Le lettere singole (diverse da *i* e *a*) sono artefatti di tokenizzazione e vengono anch'esse filtrate.

Esempi di testo generato (seed=42, 40 parole):

- *Stile Twitter*: frasi brevi, emotivamente cariche, con ripetizioni tipiche dei social.

- *Stile letterario*: frasi più lunghe, con lessico formale e costruzioni sintattiche elaborate tipiche di Austen.

7 Perplexity Cross-Domain

7.1 Configurazione dell'esperimento

L'idea è usare la perplexity come classificatore: dato un testo, il modello che gli assegna **perplexity più bassa** è quello del dominio di appartenenza.

Si usano 8 frasi di test (4 da Twitter, 4 letterarie) e si calcola la perplexity con entrambi i modelli Laplace. Poiché i vocabolari hanno dimensioni molto diverse, si normalizza dividendo per la dimensione del vocabolario:

Classificazione tramite perplexity normalizzata

```

1  for nome, frase in frasi_test.items():
2      tokens_frase = word_tokenize(frase)
3      pp_tw = modello_tweet_lp.perplexity(tokens_frase)
4      pp_lett = modello_lett_lp.perplexity(tokens_frase)
5
6      # Normalizza per la dimensione del vocabolario
7      pp_tw_n = pp_tw / len(modello_tweet_lp.vocab)
8      pp_lett_n = pp_lett / len(modello_lett_lp.vocab)
9
10     verdetto_atteso = "TW" if nome.startswith("tw") else "LT"
11     verdetto = "TW" if pp_tw_n < pp_lett_n else "LT"
12     corretto = "corretto" if verdetto == verdetto_atteso else "errato"

```

7.2 Limiti della perplexity come classificatore

La perplexity può risultare instabile quando i vocabolari hanno dimensioni molto diverse o quando le frasi di test contengono parole rare. Per questo motivo si è implementato un classificatore alternativo basato sulla log-likelihood.

8 Classificatore con Log-Likelihood

Calcolo della log-likelihood di una sequenza

```

1  def log_likelihood(modello, tokens):
2      """
3      Somma dei log P(w_i | w_{i-1}).
4      Più alta (meno negativa) = il modello conosce meglio il testo.
5      """
6      score = 0.0
7      for i in range(1, len(tokens)):
8          contesto = [tokens[i-1]]
9          parola = tokens[i]
10         prob = modello.score(parola, contesto)
11         if prob > 0:
12             score += math.log(prob)
13         else:

```

```

14         score += math.log(1e-10) # floor per parole mai viste
15     return score
16
17 def classifica_loglik(testo, mod_tw, mod_lt):
18     tokens = word_tokenize(testo.lower())
19     ll_tw = log_likelihood(mod_tw, tokens)
20     ll_lt = log_likelihood(mod_lt, tokens)
21     # vince il modello con log-likelihood piu' alta
22     verdetto = "TWITTER" if ll_tw > ll_lt else "LETTERARIO"
23     return verdetto, ll_tw, ll_lt

```

Il classificatore assegna il testo al modello con log-likelihood più alta (meno negativa). Questo approccio è più robusto della perplexity normalizzata perché:

- Non dipende dalla lunghezza della frase.
- Non è influenzato dalla dimensione del vocabolario.
- Un floor di 10^{-10} (invece di $P = 0$) evita i valori $-\infty$.

9 MLE vs. Laplace: confronto diretto

Confronto MLE e Laplace su parola mai vista

```

1 parola_rara = "xyzword"
2 print(modello_tweet.score(parola_rara, ['i'])) # MLE -> 0.0
3 print(modello_tweet_lp.score(parola_rara, ['i'])) # Laplace -> valore > 0
4
5 frase_rara = word_tokenize("i xyzword lol")
6 try:
7     pp_mle = modello_tweet.perplexity(frase_rara)
8 except:
9     pp_mle = float('inf') # MLE -> perplexity infinita
10 pp_lap = modello_tweet_lp.perplexity(frase_rara)

```

Tabella 2: Confronto MLE vs. Laplace su frase con parola mai vista

Metrica	MLE	Laplace
$P(\text{xyzword} \mid i)$	0.0	> 0
Perplexity su frase rara	∞	valore finito

Il MLE è inutilizzabile per calcolare la perplexity su frasi con parole fuori vocabolario, mentre Laplace garantisce sempre un valore finito.

10 Confronto $n = 1, 2, 3$

Confronto della perplexity al variare di n

```
1 frase_tw = word_tokenize("i love you so much thank you")
2 frase_lt = word_tokenize("she had been very well disposed")
3
4 for n_test in [1, 2, 3]:
5     tr_tmp, voc_tmp = padded_everygram_pipeline(n_test, [tokens_tweet])
6     mod_tmp = Laplace(n_test)
7     mod_tmp.fit(tr_tmp, voc_tmp)
8
9     pp_tw = mod_tmp.perplexity(frase_tw)
10    pp_lt = mod_tmp.perplexity(frase_lt)
11    print(f"n={n_test}: PP Twitter={pp_tw:.2f}, PP Letterario={pp_lt:.2f}")
```

All'aumentare di n :

- Il modello cattura dipendenze a più lungo raggio, risultando più espressivo.
- La perplexity su frasi del dominio corretto tende a diminuire (il modello conosce meglio il contesto).
- Il rischio di **data sparsity** aumenta: con $n = 3$ molti trigrammi specifici non appaiono nel training.

11 Analisi Linguistica Comparata

11.1 Type-Token Ratio

Calcolo del TTR

```
1 ttr_tweet = len(vocab_tweet_set) / len(tokens_tweet)
2 ttr_lett  = len(vocab_lett_set) / len(tokens_lett)
```

Twitter ha un TTR maggiore: nonostante i tweet siano brevi e ripetitivi nei contenuti, il vocabolario è eterogeneo per la presenza di slang, hashtag, nomi propri e abbreviazioni. Il testo letterario, pur più lungo, usa un vocabolario relativamente ristretto e ricorrente.

11.2 Lunghezza media delle frasi

Calcolo della lunghezza media delle frasi

```
1 lung_tw = [len(word_tokenize(t)) for t in tweets_raw]
2 frasi_lt = sent_tokenize(testo_letterario_raw)
3 lung_lt = [len(word_tokenize(f)) for f in frasi_lt]
4
5 media_tw = sum(lung_tw) / len(lung_tw)
6 media_lt = sum(lung_lt) / len(lung_lt)
```

Le frasi letterarie sono circa il doppio più lunghe dei tweet: il romanzo di Austen è ricco di subordinate e costruzioni relative, mentre i tweet tendono a essere telegrafici.

12 Risultati e Discussione

Tabella 3: Riepilogo delle caratteristiche dei due domini

Caratteristica	Twitter	Letterario
Dataset	10 000 tweet	<i>Emma</i> , Austen (1816)
TTR	alto (~ 0.17)	basso (~ 0.05)
Lunghezza media frase	~ 10 token	~ 22 token
Bigrammi tipici	emozioni, auguri	titoli, azioni
Smoothing consigliato	Laplace	Laplace
Classificatore migliore	Log-likelihood	Log-likelihood

12.1 Osservazioni principali

1. **I modelli N-gram riflettono lo stile del corpus:** il testo generato da ciascun modello è chiaramente riconoscibile come appartenente al dominio di addestramento.
2. **La perplexity da sola non basta come classificatore:** la differenza di dimensione del vocabolario e la presenza di parole OOV la rendono instabile; la log-likelihood è più affidabile.
3. **Laplace è indispensabile per la robustezza:** MLE è ottimo per stimare probabilità su dati visti, ma non può gestire frasi di test con parole nuove.
4. $n = 2$ è un buon compromesso: cattura dipendenze locali senza soffrire troppo di sparsity, a differenza di $n = 3$.
5. **Twitter è linguisticamente più vario ma meno strutturato:** TTR alto, frasi brevi, bigrammi emotivi. Austen ha TTR basso ma struttura sintattica ricca.

13 Conclusioni

Il laboratorio ha dimostrato come i modelli linguistici basati su N-grammi siano in grado di catturare differenze stilistiche significative tra due domini molto distanti. Il modello addestrato su Twitter genera testo informale e ripetitivo, mentre quello addestrato su Austen produce frasi più formali e articolate.

Il classificatore basato sulla log-likelihood si è rivelato più robusto della perplexity per la classificazione di dominio, grazie alla sua indipendenza dalla dimensione del vocabolario. Le tecniche di smoothing (in particolare Laplace) sono essenziali per rendere i modelli applicabili a testi reali che contengono inevitabilmente parole fuori vocabolario.